

C02 AT-2

Machine Learning Debugging Tasks - Theory and Programming

1. K-Means Clustering Debugging

Theory

Problem: K-Means may produce inconsistent clusters because of random centroid initialization, wrong number of clusters (K), outliers, and different feature scales.

Debugging & Optimization

- Use K-Means++ for better centroid initialization.
- Normalize/Standardize features.
- Remove outliers.
- Use Elbow Method and Silhouette Score to choose K.

Python Example

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

X = load_iris().data
X = StandardScaler().fit_transform(X)
model = KMeans(n_clusters=3, init='k-means++', random_state=42)
model.fit(X)
print(model.labels_)
```

2. SVM Debugging

Theory

Problem: SVM underperforms on nonlinear datasets due to wrong kernel, poor tuning of C and gamma, or unscaled features.

Debugging & Optimization

- Select suitable kernel (RBF for nonlinear data).
- Tune C and gamma using GridSearchCV.
- Apply feature scaling.

Python Example

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
X = StandardScaler().fit_transform(X)
model = SVC(kernel='rbf', C=1, gamma='scale')
model.fit(X, y)
print(model.score(X, y))
```

3. RNN Debugging

Theory

Problem: Standard RNNs struggle with long-term dependencies because of vanishing gradients and limited memory.

Solutions

- Replace RNN with LSTM or GRU.
- Use Attention Mechanism.
- Apply Gradient Clipping and Adam optimizer.

Python Example (LSTM)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential()
model.add(LSTM(64, input_shape=(10,5)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
model.summary()
```